

Computing Over the Sky: Joint UAV Trajectory and Task Offloading Scheme Based on Optimization-Embedding Multi-Agent Deep Reinforcement Learning

Xuanheng Li^{ID}, *Member, IEEE*, Xinyang Du^{ID}, *Student Member, IEEE*,
Nan Zhao^{ID}, *Senior Member, IEEE*, and Xianbin Wang^{ID}, *Fellow, IEEE*

Abstract—Unmanned aerial vehicle (UAV)-assisted mobile edge computing (MEC) has emerged to support computation-intensive tasks in 6G systems. Since the battery capacity of a UAV is limited, to serve as many users as possible, a joint design on UAV trajectory and offloading strategy with consideration for service fairness is essential to provide energy-efficient computation offloading to the users in UAV-MEC networks. Unfortunately, such a joint decision-making problem is not straightforward due to various task types required from users and various functionalities of different UAVs enabled by different application programs. Considering the above issues, we take energy efficiency and service fairness as the objective, and propose a Multi-Agent Energy-Efficient joint Trajectory and Computation Offloading (MA-ETCO) scheme. To adapt to dynamic demands of users, we develop an optimization-embedding multi-agent deep reinforcement learning (OMADRL) algorithm. Each UAV autonomously learns the trajectory control decision based on MADRL to adapt to dynamic demands. Then, it will obtain the optimal computation offloading decision by solving a mixed-integer nonlinear programming problem. The computation offloading result, in turn, will be used as an indicator to guide UAVs' trajectory design. Compared to relying solely on deep reinforcement learning, such an optimization-embedding way reduces action space dimension and improves convergence efficiency.

Index Terms—Unmanned aerial vehicle, mobile edge computing, computation offloading, trajectory control, reinforcement learning.

I. INTRODUCTION

WITH the rapid development of 5G beyond and 6G technologies, more and more computationally intensive

applications have emerged, such as virtual reality and facial recognition. However, most mobile terminal users will find it difficult to handle these computationally intensive tasks due to their resource constraints [2]. In this context, mobile edge computing (MEC) has emerged as a promising solution to help resource-limited users accomplish these computation tasks with low latency and energy consumption. Consequently, users can offload computation-intensive tasks to nearby MEC nodes for computation, also known as computation offloading [3], and thus be relieved from heavy computational workloads [4], [5], [6].

To collect and fulfill computation tasks produced by widely distributed users more efficiently, many recent works have considered the combination of UAVs and MEC [7]. Due to the high mobility and easy deployment of MEC-mounted UAVs, they can fly over a whole service area and dynamically cover hotspot areas to flexibly provide computation offloading service to users. Since the communication links between UAVs and users usually have line of sight (LoS) conditions with reduced distance, users can offload their tasks with lower latencies. To maximize computation offloading efficiency for such a UAV-assisted network, two key issues need to be well addressed, namely, UAV trajectory control and computation offloading decision.

A. UAV Trajectory Control

In general, the spatial distribution of users is uneven, and computation demand and quality of service (QoS) requirement of each user may change dynamically. Hence, it is necessary to well control the trajectory of each UAV so that it can fully utilize its flexibility to cover hot spots at different positions to satisfy as many distributed demands as possible with QoS guarantee. To achieve that, three problems need to be considered: 1) *Various Functionalities of Different UAVs*. The tasks produced by different users are usually different [6], [8], such as, face recognition, image classification, virtual reality, and so on. In order to accomplish these various tasks, different UAVs carrying different application programs will act as MEC servers to provide computation offloading services. Therefore, it is necessary to achieve a good match through trajectory control to enable each UAV to serve its corresponding tasks according to its functionality; 2) *UAV Energy*. Different flying

Manuscript received 15 December 2022; revised 17 May 2023, 16 July 2023, 6 September 2023, and 6 October 2023; accepted 31 October 2023. Date of publication 8 November 2023; date of current version 19 March 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62271100, and in part by the Dalian Talents Innovation Support Program under Grant 2021RQ053. An earlier version of this paper was presented at IEEE ICC'23 [DOI: 10.1109/ICC45041.2023.10278822]. The associate editor coordinating the review of this article and approving it for publication was C.-H. Lee. (*Corresponding author: Xuanheng Li.*)

Xuanheng Li, Xinyang Du, and Nan Zhao are with the School of Information and Communication Engineering, Dalian University of Technology, Dalian 116024, China (e-mail: xhli@dlut.edu.cn; dxy01@mail.dlut.edu.cn; zhaonan@dlut.edu.cn).

Xianbin Wang is with the Department of Electrical and Computer Engineering, Western University, London, ON N6A 5B9, Canada (e-mail: xianbin.wang@uwo.ca).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCOMM.2023.3331029>.

Digital Object Identifier 10.1109/TCOMM.2023.3331029

0090-6778 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

speeds will lead to different energy consumptions. Considering the limited on-board battery of each UAV, a sophisticated trajectory control strategy needs to be designed to achieve an energy-efficient path; 3) *Service Fairness*. Regarding computation offloading as a service, achieving service fairness implies that all users get a similar number of times of service from UAVs. To avoid the situation that some users may be frequently served while others are neglected, the fairness issue should be considered during the trajectory control of each UAV to guide them to serve as many users as possible and prevent some users from being ignored for a long time.

B. User Task Offloading Decision

When a UAV flies to a specific location, it will receive offloading requests from the users within its coverage. Then it needs to make computation offloading decisions for them, which involve admission control and resource allocation: 1) *Admission Control*. In general, according to different QoS requirements, tasks to be offloaded from users can be divided into two categories, namely, delay-sensitive tasks, e.g., autonomous driving and electric grid control and delay-tolerant tasks, e.g., fleet management and logistics, as well as air and water quality monitoring [9]. For these tasks with different delay requirements, UAVs choose different offloading modes. As for a delay sensitive task, if it is admitted, it needs to be computed by a UAV to guarantee its delay requirement. As for a delay tolerant task, it can be relayed to a base station to save limited UAV computation resources. Due to limited onboard communication and computation resources, which offloading requests from users are admitted need to be carefully determined with consideration for all QoS requirements, offloading mode, and function matching; 2) *UAV Resource Allocation*. To accomplish offloaded delay sensitive tasks, each UAV needs to well schedule its limited computation resources to guarantee their latency requirements. Thus, for each UAV, how to compute as many delay sensitive tasks as possible with its limited computation resources is another issue to address.

Determining UAV trajectory control and computation offloading decisions are two closely correlated problems. Specifically, different trajectories will allow a UAV to serve different areas with different demands and QoS requirements, which affect its decision on admission control and computation resource allocation. Besides, network utility after executing computation offloading decision can reflect whether the obtained trajectory control decision is good or not, which will be used to guide trajectory control.

Many recent works have investigated UAV-assisted MEC networks [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28] based on optimization methods to obtain the best trajectory control decisions and computation offloading decisions. To make UAVs well adapt to the dynamic environment and autonomously obtain the best strategies, some works have adopted reinforcement learning approaches to achieve the optimal decisions [29], [30], [31], [32], [33], [34], [35], [36], [37], [38]. However, they relied on reinforcement learning to learn all the best strategy/policy, which may lead to a large

action space dimension with low convergence efficiency when the strategies are very complex.

To overcome the above challenges while improving energy efficiency and achieving service fairness, we propose a Multi-Agent Energy-Efficient joint Trajectory and Computation Offloading scheme, named MA-ETCO scheme, for a UAV-assisted MEC network by taking various functionalities of different UAVs and various task types required from users into account. We note that only trajectory control decisions are influenced by dynamic task demands, while computation offloading decisions can be obtained based on the current observation by an optimization method. Hence, unlike existing works that relied solely on reinforcement learning approaches, we propose a hybrid approach by embedding an optimization method into the multi-agent deep reinforcement learning approach to determine corresponding decisions respectively, called Optimization-embedding Multi-Agent Dueling Deep Q Network (OMADDQN) algorithm. Specifically, we utilize the MADRL method to capture the features of demands and user distributions to learn the trajectory control decision of each UAV. When a UAV moves to a certain point, we formulate its computation offloading decision as an Mixed-Integer Nonlinear Programming (MINLP) problem to obtain the optimal admission control and computation resource allocation strategies based on current observations, which is used as an indicator to guide the MADRL-based trajectory control design. Such an optimization-embedding way can reduce action space dimension. Furthermore, collision avoidance is an important issue in many multi-agent scenarios, such as robotics [39], [40]. As for the UAV-assisted MEC system in our work, such an issue also exists when making trajectory control decisions. Hence, to avoid the flight collision among different UAVs, we design a collision avoidance action adjustment mechanism to adjust the output action of OMADDQN network. In our preliminary work [1], we designed an MADRL-based method to achieve joint design but did not consider energy efficiency issue, and all UAVs were assumed to be equipped with the same functionality. The main contributions of this paper are summarized below.

- To our best knowledge, this is the first work to jointly design UAV trajectory and computation offloading strategy while considering all issues on service fairness, various functionalities of different UAVs and various task types requiring different application programs to support them in a UAV-assisted MEC network. Taking energy efficiency and service fairness as the objective, we propose an MA-ETCO scheme that enables UAVs to dynamically cover hot spots and fulfill computation tasks with QoS guarantee.
- To deal with dynamic task demands and achieve cooperation between UAVs, we develop a multi-agent deep reinforcement learning approach with a collision avoidance action adjustment (CAAA) mechanism for the proposed scheme. It can help UAVs adapt to a dynamic environment, achieve group intelligence, and autonomously obtain the optimal joint decision.
- Instead of relying solely on DRL approaches to obtain all decisions, we fully utilize the advantages of

optimization methods and DRL approaches, and propose an Optimization-Embedding Multi-Agent Dueling Deep Q Network algorithm. Based on MADDQN, each UAV autonomously learns its trajectory control decision in a dynamic environment to widely cover hot spots. After arriving at a certain location, the optimal computation offloading decision will be obtained by an optimization method. The obtained computation offloading result is used to guide the MADRL-based trajectory control. Such a hybrid design can effectively reduce action space dimension and improve convergence efficiency.

The remainder of this paper is organized as follows. In Section II, we review the related works to put up our work. In Section III, the system model is presented for practical application scenarios. The OMADDQN algorithm is introduced in Section IV. In Section V, we discuss the simulation results. Finally, the conclusion is drawn in Section VI.

II. RELATED WORKS

Due to the explosive growth of computation requirements in 6G era, many works have used UAVs as flying or mobile servers. A basic technology to implement UAV-assisted MEC is UAV virtualization [41], [42], through which a MEC-mounted UAV with different functionalities can be realized.

The works on UAV-assisted MEC networks [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28] mainly focused on three research issues, namely, UAV deployment or trajectory design, offloading decision, and resource management. Many excellent works have proposed different effective static UAV deployment methods to provide MEC services to ground users from different perspectives, such as wide coverage [10], low energy consumption [11], [12], [19], [20], low time cost [11], [13], [14], [15], high data rate [12], [16], [17], interference avoidance [18], [20], high resource utilization [17], [20], and security assurance [19].

Considering the scattered distribution of ground users and dynamic hotspot areas, many studies [21], [22], [23], [24], [25], [26], [27], [28] replaced UAV deployment with UAV trajectory control by fully utilizing the mobility of UAVs to dynamically cover their service area. Taking different QoS metrics into account, such as delay requirements [21], queue length of tasks [22], [23], SINR [24], energy consumption [22], [23], [24], [25], [26], [27], [28] and endurance [27], they developed different novel joint trajectory and offloading schemes based on successive convex approximation strategy [21], [26], Lyapunov optimization [22], [23], three-stage iterative algorithm [24], dynamic programming-based optimization method [25], Dinkelbach method [26], block coordinate descent algorithm [27], and alternating iterative algorithms [28], respectively.

The above works relied on optimization approaches to design the joint strategy. In practice, since users demands and their distributions are usually time-varying, there are many recent UAV-assisted MEC works focused on reinforcement

learning algorithms to let UAVs autonomously explore an uncertain environment and effectively learn decisions [29], [30], [31], [32], [33], [34], [35], [36], [37], [38]. Taking the long-term energy efficiency of all UAVs as the objective, Li et al. [29] proposed a novel triple-learner-based reinforcement learning (TLRL) approach to jointly optimize the trajectory planning, energy renewal, and application placement of each UAV. To minimize both communication and computation latency, Liu et al. [30] utilized DQN and DDPG algorithms to optimize both the trajectory and virtual machine configuration of each UAV. In [31], Wan et al. proposed an online edge processing scheduling algorithm to change the task processing scheduling decision of UAV according to the real-time data rate to realize computing efficiency. Considering the limited UAV resources, Wang et al. [32] used solar energy to charge each UAV while providing computing services for wireless users, finally minimizing the total service cost, while Liu et al. [33] designed a UAV cluster collaborative computation offloading scheme to maximize the long-term utility of networks while avoiding interference. To handle a high-dimensional continuous action space, Zhao et al. [34] exploited a twin delayed deep deterministic policy gradient (TD3PG) algorithm to obtain the joint strategy of trajectory design, task allocation, and power management. Utilizing the MADDPG algorithm, Peng and Shen [35] investigated multi-dimensional resource management for UAV-assisted vehicular networks to maximize the number of offloaded tasks, and Wang et al. [36] took energy efficiency as an objective to jointly optimize the trajectory and computation offloading decision of each UAV while considering service fairness and collision avoidance. Taking service cost and task execution delay as objectives, Seid et al. [37] proposed a DDPG scheme to generate an optimal computation offloading policy for devices. Liu et al. [38] proposed an attention-based MAPPO algorithm to pursue the optimal computation offloading policy of UAVs to minimize the weighted energy consumption while considering service fairness.

Although these works developed different DRL approaches to achieve optimal UAV decisions in a dynamic environment, they did not jointly consider fairness, various functionalities of different UAVs, various task types, or energy efficiency. The specific differences between our work and existing related works are listed in Table I. Furthermore, these works obtained all decisions based on reinforcement learning algorithms. However, since some decisions are not affected by environmental dynamics, they may not be necessary to be determined by reinforcement learning approaches, which may lead to a large action space dimension. Different from these works, taking various functionalities of different UAVs and various task types into account, we jointly design UAV trajectory control and computation offloading strategies to maximize fairness-based energy efficiency while satisfying the delay requirements of users. Considering the different impacts of environmental dynamics on different decisions, we develop a hybrid approach by embedding an optimization method into the multi-agent deep reinforcement learning approach, which can reduce action space dimension and improve convergence efficiency.

TABLE I
COMPARISON BETWEEN OUR WORK AND EXISTING RELATED WORKS

Reference	Service Fairness	UAV Group Functionalities	Various Task Types	Solution			
				Optimization	RL	DRL	MADRL
[21]–[25]				✓			
[26]			✓	✓			
[27]		✓		✓			
[29]		✓	✓		✓		
[30]			✓			✓	
[31], [32]						✓	
[33]–[35]							✓
[36]–[38]	✓						✓
Our work	✓	✓	✓	✓			✓

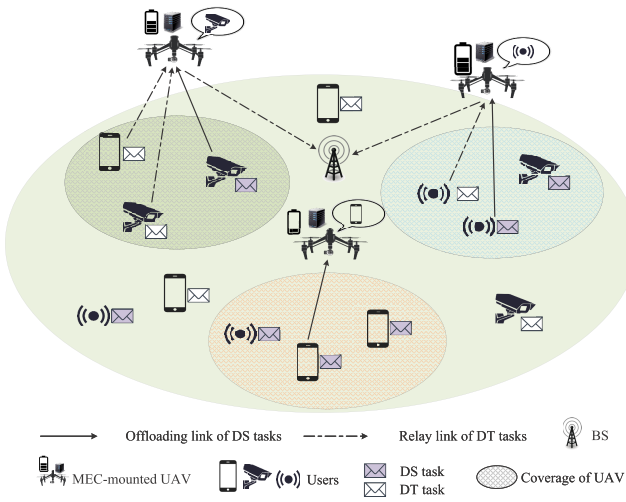


Fig. 1. System model of a UAV-assisted MEC network.

III. SYSTEM MODEL

A. Network Model

We investigate a UAV-assisted MEC network as shown in Fig. 1. Specifically, a group of M UAVs are deployed in a network, acting as MEC nodes to offer computation offloading services for N fixed users. According to different QoS requirements and demands of users, their offloaded tasks will be either computed by a corresponding UAV or relayed to a base station (BS).

We assume time synchronization among all UAVs [43] and all of them follow a flight-then-service (F-S) procedure in each time slot, as shown in Fig. 2. During the flight phase T_1 , UAV m will hover or fly from location $\mathbf{l}_{m,t-1} = [x_{m,t-1}, y_{m,t-1}, H]$ to location $\mathbf{l}_{m,t} = [x_{m,t}, y_{m,t}, H]$ at speed $v_{m,t}$. When UAV m arrives at its serving areas after the flight phase T_1 , it needs to decide which tasks within its coverage to admit. The coverage radius of each UAV is R^{cov} , and the set of users within the coverage of UAV m in time slot t is denoted as $\mathcal{N}_{m,t}$. If the coverage areas of two UAVs overlap, tasks in the overlapping area will be served by the closer UAV.

As for user n , we assume that it produces one task in each time slot t with data volume $Q_{n,t}$ and $F_{n,t}$ CPU cycles are required to calculate one bit. In different time slots, the task data volumes and requirements of user n are different

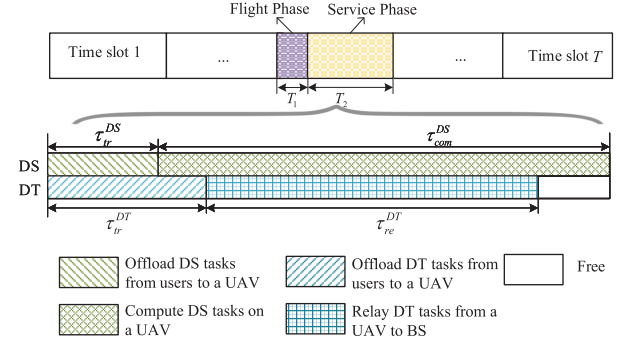


Fig. 2. The flight-then-service procedure of each UAV in a time slot.

and dynamic. According to characteristics of different applications, tasks are divided into two categories, namely, delay sensitive (DS) and delay tolerant (DT). The delay sensitive task produced by user n has a specific delay requirement $\hat{T}_n$, which needs to be executed with a latency lower than it. We assume that there are K types of delay sensitive tasks [6] and K types of corresponding UAVs. Each UAV carries one kind of application programs to compute a certain type of delay sensitive tasks.¹ To guarantee the QoS requirement of each delay sensitive task, it can only be offloaded to a UAV with corresponding application programs or computed by the user locally. We use a binary variable $z_{m,n,t}^{\text{DS}} = \{0, 1\}$ to denote whether or not UAV m admits the delay sensitive task produced by user $n \in \mathcal{N}_{m,t}$. Since delay tolerant tasks do not have strict time delay requirements, they will be relayed to a BS by UAVs to save limited UAV computation resources.

Since relaying delay tolerant tasks to a BS also consumes UAV energy and bandwidth resources, it is unreasonable for each UAV to directly accept all of them to relay to a BS. Hence, we define a binary variable $z_{m,n,t}^{\text{DT}} = \{0, 1\}$ to indicate whether or not UAV m relays delay tolerant task n to a BS. We assume that the maximum number of tasks admitted by a UAV during each time slot is z^{max} . After admission control, each UAV needs to allocate computation resources to admitted delay sensitive tasks to satisfy their delay requirements. As shown in Fig. 2, the delay for executing an admitted delay sensitive task contains offloading delay and

¹Since the on-board resources of each UAV is limited, without loss of generality, we consider a case that each UAV only carries one kind of application programs to support one type of delay sensitive tasks. It is noteworthy that although our scheme is developed for such a case, it can be extended to the case where each UAV carries various functions.

computation delay, and that for an admitted delay tolerant task contains offloading delay and relay delay. The F-S procedure will be repeated until the energy of each UAV is less than the threshold $E^{\min}$, and then it will return to the charging station. The threshold $E^{\min}$ is the minimal energy required for UAVs to return to the charging station from all areas. Some main notation definitions are shown in Table II.

B. UAV Flying Power Model

During the flight phase, UAV m horizontally flies from the previous location $\mathbf{l}_{m,t-1}$ to the next location $\mathbf{l}_{m,t}$ at the speed $v_{m,t}$ or hovers.

If UAV m decides to fly horizontally, the horizontal flying power is expressed as

$$P_{m,t}^{\text{level}} = \frac{M_a^2}{\sqrt{2}\rho A} \cdot \frac{1}{\sqrt{\|v_{m,t}\|^2 + \sqrt{\|v_{m,t}\|^4 + \left(\frac{M_a}{2\rho A}\right)^2}}}, \quad (1)$$

where M_a is the quality of each UAV, ρ is air density, A is blade area without helical wing. Besides, the power consumed to overcome air resistance is given by

$$P_{m,t}^{\text{drag}} = \frac{1}{8} C_{\text{air}} \rho A \|v_{m,t}\|^3, \quad (2)$$

where C_{air} is the air drag coefficient related to helical wing blade shape [44].

If UAV m decides to hover, the hovering power is

$$P_{m,t}^{\text{hover}} = \frac{M^{3/2}}{\sqrt{2}\rho A}. \quad (3)$$

The overall flight power of UAV during the flight phase T_1 is calculated as

$$P^{\text{fly}} = \begin{cases} P_{m,t}^{\text{level}} + P_{m,t}^{\text{drag}}, & \text{horizontal flying,} \\ P_{m,t}^{\text{hover}}, & \text{hovering.} \end{cases} \quad (4)$$

C. Communication Model

After the flight phase, UAV m will arrive at the next location $\mathbf{l}_{m,t}$ and start to collect data from admitted tasks based on the ground-to-air (G-A) link [45]. The LoS connection probability of the G-A link is expressed as

$$\delta_{m,n,t} = \frac{1}{1 + a \exp(-b(\arctan(\frac{H}{d_{m,n,t}}) - a))}, \quad (5)$$

where a and b are environmental parameters, $d_{m,n,t}$ denotes the horizontal distance between UAV m and user n , as well as H denotes the height of UAV. Note that $d_{m,n,t}$ is calculated as

$$d_{m,n,t} = \sqrt{(x_{m,t} - x_n)^2 + (y_{m,t} - y_n)^2}. \quad (6)$$

The average pass loss $l_{m,n,t}$ (in dB) between UAV m and user n contains the free space pathloss and excessive pathloss [46], which is expressed as

$$l_{m,n,t} = l^{\text{free}} + \delta_{m,n,t} \cdot \eta^{\text{LoS}} + (1 - \delta_{m,n,t}) \cdot \eta^{\text{NLoS}}, \quad (7)$$

TABLE II
LIST OF MAIN NOTATIONS

Notation	Description
$n, N, \mathcal{N}$	The index, number and the set of users
$m, M, \mathcal{M}$	The index, number and the set of UAVs
K	Types of delay sensitive tasks
$D_{n,t}$	Data volume of the task produced by user n in time slot t
R^{cov}	Coverage radius of UAV
$\mathcal{N}_{m,t}$	The set of users within coverage of UAV in time slot t
$d_{m,n,t}$	Horizontal distance between UAV m and user n .
$z_{m,n,t}^{\text{DS}}$	Admission control of UAV m for delay sensitive task
$z_{m,n,t}^{\text{DT}}$	Admission control of UAV m for delay sensitive task
$\eta_{m,t}, v_{m,t}$	Flying angle and speed of UAV m in time slot t
$l_{m,n,t}$	Average pass loss (in dB) between UAV m and user n
$r_{m,n,t}$	Average transmission rate from user n to UAV m
$r_{m,b,t}$	Average transmission rate from UAV m to a BS
$f_t(n)$	Fairness indicator of user n in time slot t
$D^{\min}$	Minimal safe distance between UAVs
$s_{m,t}$	The state of UAV m in time slot t
$a_{m,t}$	The action of UAV m in time slot t
$r_{m,t}$	The reward of UAV m in time slot t
$F_{n,t}$	CPU cycles required to calculate one bit of task n in time slot t
$z^{\max}$	The maximum number of tasks admitted by one UAV in each time slot
$c_{m,n,t}$	Computation resources of UAV m allocated to delay sensitive task n
$E^{\min}$	The minimal energy required for a UAV to return to the charging station

where η^{LoS} and η^{NLoS} refer to the mean value of excessive pathloss, namely, the average shadowing parameters of LoS link and NLoS link, respectively, and l^{free} is expressed as

$$l^{\text{free}} = 20\log_{10}(\sqrt{d_{m,n,t}^2 + H^2}) + 20\log_{10}(f) + 20\log_{10}\left(\frac{4\pi}{c}\right), \quad (8)$$

where f and c denote the carrier frequency and speed of light, respectively.

According to (7), the average transmission rate from user n to UAV m is calculated as

$$r_{m,n,t} = B\log_2\left(1 + \frac{P_n}{10^{l_{m,n,t}/10} \cdot \sigma^2}\right), \quad (9)$$

where σ^2 is White Gaussian Noise variance, P_n is the transmission power of user n , and B denotes the channel bandwidth allocated to user n .

As aforementioned, the admitted delay tolerant tasks will be relayed to a BS. Since a UAV-to-BS channel is usually LoS-dominated, the average pass loss $l_{m,b,t}$ (in dB) between UAV m and a BS is expressed as

$$l_{m,b,t} = 20\log_{10}(\sqrt{d_{m,b,t}^2 + (H^2 - H_b^2)}) + 20\log_{10}(f) + 20\log_{10}\left(\frac{4\pi}{c}\right) + \eta^{\text{LoS}}, \quad (10)$$

where $d_{m,b,t}$ denotes the horizontal distance between UAV m and a BS, and H_b denotes the height of a BS. The achievable transmission rate of UAV-to-BS is calculated as

$$r_{m,b,t} = B\log_2\left(1 + \frac{P_m}{10^{l_{m,b,t}/10} \cdot \sigma^2}\right), \quad (11)$$

where P_m denotes the transmission power of UAV m .

D. Computation Model

As for the admitted delay sensitive task produced by user n , we denote $c_{m,n,t}$ as the CPU frequencies (in cycle/s) assigned to it in time slot t . Since $Q_{n,t}$ is the data volume and $F_{n,t}$ is the required CPU cycles to calculate one bit, the delay for computing it by UAV m is calculated as

$$\tau_{m,n,t}^{com} = \frac{Q_{n,t} \cdot F_{n,t}}{c_{m,n,t}}. \quad (12)$$

According to (9), the transmission delay from user n to UAV m in time slot t is calculated as

$$\tau_{m,n,t}^{tr} = \frac{Q_{n,t}}{r_{m,n,t}}. \quad (13)$$

The overall time to execute the delay sensitive task by UAV m is represented as

$$\tau_{m,n,t}^{DS} = \tau_{m,n,t}^{tr} + \tau_{m,n,t}^{com}. \quad (14)$$

E. Energy Consumption Model

The energy consumption of a UAV contains three main components, flying/hovering, communication, and computation, respectively.

1) *Flying and Hovering Energy Consumption:* According to (4), energy consumption during flight phase T_1 is calculated as

$$E_{m,t}^f = P_{m,t}^{fly} \cdot T_1. \quad (15)$$

After that, UAV will hover during service phase T_2 . The hovering energy is modeled as

$$E_{m,t}^h = P_{m,t}^{hover} \cdot T_2. \quad (16)$$

Hence, the total flying and hovering energy consumption during time slot t is expressed as

$$E_{m,t}^F = E_{m,t}^f + E_{m,t}^h. \quad (17)$$

2) *Computation Energy Consumption:* If UAV m decides to admit and compute the delay sensitive task produced by user n , the computation energy consumption [47] is calculated as

$$E_{m,n,t}^{DS} = \varepsilon^{MEC} \cdot c_{m,n,t}^3 \cdot \tau_{m,n,t}^{com}, \quad (18)$$

where ε^{MEC} denotes UAV effective capacitance coefficient [48], $c_{m,n,t}$ is the CPU frequencies assigned to delay sensitive task n , and $\tau_{m,n,t}^{com}$ denotes the computation time during time slot t .

3) *Communication Energy Consumption:* According to (11), if UAV m decides to relay the delay tolerant task produced by user n from UAV m to a BS, the transmission delay is expressed as

$$\tau_{m,n,b,t}^{re} = \frac{Q_{n,t}}{r_{m,b,t}}, \quad (19)$$

and the energy consumption for relaying admitted delay tolerant task n is given by

$$E_{m,n,t}^{DT} = \tau_{m,n,b,t}^{re} \cdot P_m. \quad (20)$$

F. Problem Formulation

To achieve energy efficient UAV-assisted computation offloading, we jointly design UAV trajectory control, admission control, and computation resource allocation strategies with considering delay requirements of different tasks and fairness. To reflect the number of times user n has been served from the initial time slot to time slot t , we define a fairness indicator $f_t(n)$, which is expressed as

$$f_t(n) = 1 - \frac{b_{n,t}}{I}, \quad (21)$$

where $b_{n,t}$ denotes the number of times user n has been served from time slot 1 to time slot t and I is the maximum number of time slots that a UAV flies around network.² Thus, $f_t(n)$ is inversely proportional to $b_{n,t}$. If user n has been seldom served, $f_t(n)$ is close to one. As the served frequencies of user n increase, the value of $f_t(n)$ will reduce to zero. Thus, it will be used as a weighting factor to achieve fairness.

We take the fairness-based computation offloading energy efficiency (FEE) as the objective, which is expressed as (22), shown at the bottom of the next page. Note that γ_1 and γ_2 are the bias parameters for serving delay sensitive and delay tolerant tasks, respectively. Besides, $z_{m,n,t}^{DS}$ and $z_{m,n,t}^{DT}$ are admission control decisions, and $Q_{n,t}$ is the data volume of task n in time slot t . Furthermore, $E_{m,t}^F$, $E_{m,n,t}^{DS}$, and $E_{m,n,t}^{DT}$ are energy consumptions of UAV m during time slot t . Eq. (22) indicates the amount of fairness-based offloaded task data volume (in MB) executed by UAV m over its total energy consumption (in J) in time slot t .

We formulate the MA-ETCO scheme as **P1**. The target of MA-ETCO scheme is maximizing FEE of all UAVs during a whole operation process, which is expressed as:

$$\begin{aligned} \mathbf{P1} : & \max_{\mathbf{l}, \mathbf{z}, \mathbf{c}} \sum_{t=1}^T \sum_{m=1}^M \mathbb{E}[FEE_{m,t}] \\ \text{s.t.} \quad & \text{C1} : \sum_{m=1}^M z_{m,n,t}^{DS} \leq 1, \quad \forall n, \forall t, \\ & \text{C2} : \sum_{m=1}^M z_{m,n,t}^{DT} \leq 1, \quad \forall n, \forall t, \\ & \text{C3} : \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{DS} + \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{DT} \leq z^{\max}, \quad \forall m, \forall t, \\ & \text{C4} : \tau_{m,n,t}^{DS} \leq \hat{T}_n, \quad \forall m, \forall n \in \mathcal{N}_{m,t}, \forall t, \\ & \text{C5} : \sum_{n \in \mathcal{N}_{m,t}} c_{m,n,t} \leq c^{\max}, \quad \forall m, \forall t, \\ & \text{C6} : c_{m,n,t} \leq z_{m,n,t}^{DS} \cdot c^{\max}, \quad \forall m, \forall n \in \mathcal{N}_{m,t}, \forall t, \\ & \text{C7} : \|\mathbf{l}_{m,t} - \mathbf{l}_{m',t}\| \geq D^{\min}, \quad \forall m \neq m', \forall t, \\ & \text{C8} : \|\mathbf{l}_{m,t} - \mathbf{l}_{m,t-1}\| = v_{m,t} \cdot T_1, \quad \forall m, \forall t. \end{aligned}$$

²Actually, in addition to the fairness indicator $f_t(n)$ defined here, there are also many ways to achieve fairness, such as maximizing the minimal $b_{n,t}$. However, maximizing the minimal $b_{n,t}$ depends on the served times of all users, which requires global information in a centralized learning manner. But $f_t(n)$ can be used in a distributed way because each UAV can obtain computation offloading decision depending on $f_t(n)$ of users within its coverage.

where $\mathbf{l}, \mathbf{z}, \mathbf{c}$ are decision variables that represent trajectory control of each UAV, admission control, and computation resource allocation for admitted delay sensitive tasks, respectively. Constraint C1 and constraint C2 mean each task only can be admitted by one UAV in each time slot. We use constraint C3 to constrain the number of tasks admitted by a UAV in each time slot. We set constraint C4 to guarantee the delay requirements of admitted delay sensitive tasks. Constraint C5 indicates that the computation resources allocated to admitted delay sensitive tasks cannot exceed the total computation resources of each UAV. Constraint C6 indicates that only the admitted delay sensitive tasks will be allocated computation resources of UAV. Constraint C7 is used to guarantee safe distance between UAVs. We use constraint C8 to constrain the locations of each UAV on two adjacent time slots.

For **P1**, to maximize the long-term fairness-based energy efficiency of UAVs, exact information on user demands is required. However, such information is dynamic and hardly obtainable in advance, which will cause **P1** to be intractable if we rely on traditional optimization approaches. Furthermore, **P1** is non-convex due to its fractional objective function and binary decision variables. Thus we develop a multi-agent deep reinforcement learning approach in Section IV to achieve the optimal decisions.

Note that the operation of each UAV follows an F-S procedure as shown in Fig.2, where flight phase only involves $\mathbf{l}$ and service phase is related to $\mathbf{z}$ and $\mathbf{c}$, and the constraints on the two phases are actually irrelevant, i.e., constraints (C1)–(C6) for admission control and computation resource allocation and constraints (C7)–(C8) for trajectory control. As a result, **P1** can be divided into two sequential sub-problems, i.e., trajectory control and computation offloading decision. To deal with the uncertainty of demand, trajectory control decision of each UAV can be obtained by MADRL to allow them well cover network demands cooperatively and adaptively. As for a UAV, after it executes its action, it will arrive at the next location and obtain computation offloading decisions based on its current location by taking FEE (22) as the objective. Then, fairness-based energy efficiency acts as the reward of the current time slot to guide the MADRL-based trajectory control. This is our proposed optimization-embedding MADRL algorithm. Compared to relying solely on MADRL, the proposed optimization-embedding MADRL algorithm can reduce action space dimension and improve convergence efficiency.

IV. AN OPTIMIZATION-EMBEDDING MULTI-AGENT DEEP REINFORCEMENT LEARNING SOLUTION

In this section, we will introduce the proposed Optimization-Embedding Multi-Agent Deep Reinforcement Learning algorithm, named OMADRL algorithm, which is used to address the **P1** problem. In each time slot t , UAV m observes its current state $s_{m,t}$ and obtains its action $a_{m,t}$. After UAV m

executes the chosen action, it will arrive at the next location $\mathbf{l}_{m,t}$ at the end of the flight phase. Then it will achieve optimal admission control and computation resource allocation ($\mathbf{z}_{m,t}^*, \mathbf{c}_{m,t}^*$) by an optimization method [49], [50]. Then, UAV m will obtain the next state $s_{m,t+1}$ and reward $r_{m,t}$ based on $\mathbf{z}_{m,t}^*, \mathbf{c}_{m,t}^*$, which guides UAV m to learn the trajectory control decision via the MADRL.

A. Prior Knowledge of Deep Reinforcement Learning

Deep reinforcement learning is an improvement on reinforcement learning algorithms, which solves the curse of dimensionality resulting from the continuous state space. The Deep Q Network (DQN) is a normal deep reinforcement learning algorithm that contains two deep neural networks (DNNs) with the same structure, namely, the main network and the target network. The former is used to create the Q-value and the latter is used to obtain the target Q-value. Both of them are used to calculate the loss function to update network parameters. At each time slot t , each UAV gets state s_t and obtains action a_t based on the Q-value $Q(s_t, a_t; \theta)$ from the main network. Then, it will receive the reward r_t and obtain the next state s_{t+1} . Next, the transition tuple (s_t, a_t, r_t, s_{t+1}) will be stored into replay memory. A mini-batch will be randomly sampled from the replay memory to calculate the target Q-value and minimize the loss function for training. Specifically, s_{t+1} will be used to obtain $Q'(s_{t+1}, a; \theta')$ by the target network, and the target Q-value y_t is expressed as

$$y_t = r_t + \gamma \max_{a'} Q'(s_{t+1}, a'; \theta'). \quad (23)$$

The loss function is defined as

$$L(\theta) = \mathbb{E}[(y_t - Q(s_t, a_t; \theta))^2], \quad (24)$$

which will be used to update the parameters of the main network by performing a gradient step. Besides, the parameters θ' of the target network are copied from the main network periodically.

B. The Framework of The Proposed OMADRL Algorithm

1) *State*: We denote the state of UAV m in time slot t as $s_{m,t}$, which is represented as

$$s_{m,t} = \{\mathbf{L}_{t-1}, \mathbf{F}_t, \mathbf{Q}_t, \mathbf{b}_{t-1}, E_{m,t}\}, \quad (25)$$

where $\mathbf{L}_{t-1} = \{\mathbf{l}_{1,t-1}, \mathbf{l}_{2,t-1}, \dots, \mathbf{l}_{M,t-1}\}$ denotes the location of M UAVs at the beginning of time slot t , $\mathbf{F}_t = \{F_{1,t}, F_{2,t}, \dots, F_{N,t}\}$ represents the CPU cycles required by all N users for executing one bit of its task in time slot t , $\mathbf{Q}_t = \{Q_{1,t}, Q_{2,t}, \dots, Q_{N,t}\}$ is the data volume of tasks in time slot t and $\mathbf{b}_{t-1} = \{b_{1,t-1}, b_{2,t-1}, \dots, b_{N,t-1}\}$ denotes the times that N users have been served during the previous $t-1$ time slots. Besides, $E_{m,t}$ denotes the remaining energy

$$FEE_{m,t} = \frac{\sum_{n \in \mathcal{N}_{m,t}} \gamma_1 \cdot f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t} + \sum_{n \in \mathcal{N}_{m,t}} \gamma_2 \cdot f_t(n) \cdot z_{m,n,t}^{\text{DT}} \cdot Q_{n,t}}{E_{m,t}^F + \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DS}} \cdot E_{m,n,t}^{\text{DS}} + \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DT}} \cdot E_{m,n,t}^{\text{DT}}}. \quad (22)$$

of UAV m at the beginning of time slot t , which is expressed as

$$E_{m,t} = E_{m,t-1} - \sum_{n \in \mathcal{N}_{m,t-1}} z_{m,n,t-1}^{\text{DS}} \cdot E_{m,n,t-1}^{\text{DS}} - \sum_{n \in \mathcal{N}_{m,t-1}} z_{m,n,t-1}^{\text{DT}} \cdot E_{m,n,t-1}^{\text{DT}} - E_{m,t-1}^F. \quad (26)$$

If $E_{m,t} \leq E^{\min}$, UAV m will fly back to the charging station before its battery is exhausted.

2) *Action*: Based on state $s_{m,t}$, UAV m will choose an action. There are three decision variables l, z, c in the MA-ETCO scheme, where trajectory control l is the decision made through MADRL, including flying direction and speed. The action is defined as

$$a_{m,t} = \{\eta_{m,t}, v_{m,t}\}, \quad (27)$$

where $\eta_{m,t}$ represents the flight directions {North, South, East, West, Northeast, Northwest, Southeast, Southwest}, and $v_{m,t}$ is the speed ranging within $[0, v_{\max}]$, which is discretized into four levels.

3) *Optimization-Based Reward*: After executing action, UAV m arrives at the next location. Then the optimal admission control $z_{m,t}^*$ and optimal computation resource allocation $c_{m,t}^*$ will be obtained by an optimization approach. The optimization result represents computation offloading performance and will be used to guide trajectory control design under MADRL. According to (22), we define the reward of UAV m in time slot t as

$$r_{m,t} = FEE_{m,t}(z_{m,t}^*, c_{m,t}^*). \quad (28)$$

Next, we will introduce how to determine the optimal admission control $z_{m,t}^*$ and computation resource allocation $c_{m,t}^*$ based on two MINLP problems for delay sensitive tasks and delay tolerant tasks, respectively.

C. Optimization-Based Computation Offloading Decision

With regard to computation offloading decision making problem, we formulate an MINLP problem, called **COD-DS**, to obtain the optimal admission control and computation resource allocation decisions for delay sensitive tasks, i.e., $(z_{m,t}^{\text{DS}})^*$ and $c_{m,t}^*$. Also, we formulate an MINLP problem, called **COD-DT**, to obtain the optimal admission control decisions for delay tolerant tasks, i.e., $(z_{m,t}^{\text{DT}})^*$. The admission control in both **COD-DS** and **COD-DT** problems interact with each other because the maximum number of admitted tasks is limited. To fully utilize the computation resources of each UAV, we first solve the **COD-DS** problem to obtain $(z_{m,t}^{\text{DS}})^*$ for delay sensitive tasks and then determine the admitted delay tolerant tasks $(z_{m,t}^{\text{DT}})^*$ under the admission limitation.

According to **P1**, the **COD-DS** problem for the delay sensitive tasks is formulated as

$$\begin{aligned} \text{COD-DS : } \max_{z^{\text{DS}}, c} & \frac{\sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t}}{\sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DS}} \cdot E_{m,n,t}^{\text{DS}}} \\ \text{s.t. } \text{C1 : } & \sum_{m=1}^M z_{m,n,t}^{\text{DS}} \leq 1, \quad \forall n \in \mathcal{N}_{m,t}, \end{aligned}$$

$$\text{C3_1 : } \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DS}} \leq z^{\max},$$

$$\text{C4 : } \tau_{m,n,t}^{\text{DS}} \leq \hat{T}_n, \quad \forall n \in \mathcal{N}_{m,t},$$

$$\text{C5 : } \sum_{n \in \mathcal{N}_{m,t}} c_{m,n,t} \leq c^{\max},$$

$$\text{C6 : } c_{m,n,t} \leq z_{m,n,t}^{\text{DS}} \cdot c^{\max}, \quad \forall n \in \mathcal{N}_{m,t}.$$

As for the delay tolerant tasks, the **COD-DT** problem is formulated as

$$\begin{aligned} \text{COD-DT : } \max_{z^{\text{DT}}} & \frac{\sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DT}} \cdot Q_{n,t}}{\sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DT}} \cdot E_{m,n,t}^{\text{DT}}} \\ \text{s.t. } \text{C2 : } & \sum_{m=1}^M z_{m,n,t}^{\text{DT}} \leq 1, \quad \forall n \in \mathcal{N}_{m,t}, \\ \text{C3_2 : } & \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DT}} \leq z^{\max} - \sum_{n \in \mathcal{N}_{m,t}} (z_{m,n,t}^{\text{DS}})^*. \end{aligned}$$

where $(z_{m,n,t}^{\text{DS}})^*$ is the result by solving the **COD-DS** problem. Both **COD-DS** problem and **COD-DT** problem are non-convex. Since they have similar formulations, next, we present the solution to **COD-DS** problem, and that to **COD-DT** problem can be easily obtained in a similar way.

Considering the fractional objective function of **COD-DS** problem, we employ a Dinkelbach Iterative Method [48] to handle it. First, we define $\lambda_{m,t}$ as the fairness-based energy efficiency of UAV m for computing delay sensitive tasks in time slot t , given by

$$\lambda_{m,t} = \frac{\sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t}}{\sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DS}} \cdot E_{m,n,t}^{\text{DS}}}. \quad (29)$$

Then, we rewrite the fractional objective function (29) into a subtractive form, and define a function based on $\lambda_{m,t}$, which is given by

$$\begin{aligned} \phi(\lambda_{m,t}) = & \sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t} \\ & - \lambda_{m,t} \cdot \sum_{n \in \mathcal{N}_{m,t}} z_{m,n,t}^{\text{DS}} \cdot E_{m,n,t}^{\text{DS}}. \end{aligned} \quad (30)$$

According to (12) and (18), we rewrite $\phi(\lambda_{m,t})$ as

$$\begin{aligned} \phi(\lambda_{m,t}) = & \sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t} \\ & - \lambda_{m,t} \cdot \sum_{n \in \mathcal{N}_{m,t}} \varepsilon^{\text{MEC}} \cdot z_{m,n,t}^{\text{DS}} \cdot c_{m,n,t}^2 \cdot Q_{n,t} \cdot F_{n,t}. \end{aligned} \quad (31)$$

To tackle the quadratic variable $c_{m,n,t}^2$, we introduce an auxiliary variable $\mu_{m,n,t} = c_{m,n,t} \cdot c^{\max}$. Such a substitution will not affect the optimization result, and we rewrite (31) as

$$\begin{aligned} \bar{\phi}(\lambda_{m,t}) = & \sum_{n \in \mathcal{N}_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t} \\ & - \lambda_{m,t} \cdot \sum_{n \in \mathcal{N}_{m,t}} \varepsilon^{\text{MEC}} \cdot z_{m,n,t}^{\text{DS}} \cdot \mu_{m,n,t} \cdot Q_{n,t} \cdot F_{n,t}. \end{aligned} \quad (32)$$

To further deal with the multiplication of variables in (32), we introduce an auxiliary variable $\tilde{\mu}_{m,n,t} = z_{m,n,t}^{\text{DS}} \cdot \mu_{m,n,t}$. Since $\mu_{m,n,t} = c_{m,n,t} \cdot c^{\text{max}}$, we have

$$\tilde{\mu}_{m,n,t} = z_{m,n,t}^{\text{DS}} \cdot c_{m,n,t} \cdot c^{\text{max}}. \quad (33)$$

According to constraint C6 ($c_{m,n,t} \leq z_{m,n,t}^{\text{DS}} \cdot c^{\text{max}}$), we observe that only when $z_{m,n,t}^{\text{DS}} = 1$, $c^{\text{max}} \geq c_{m,n,t} \geq 0$. Otherwise, $c_{m,n,t} = 0$. Thus, we have $z_{m,n,t}^{\text{DS}} \cdot c_{m,n,t} = c_{m,n,t}$ under the constraint C6. Hence, we rewrite (33) as

$$\tilde{\mu}_{m,n,t} = c_{m,n,t} \cdot c^{\text{max}}. \quad (34)$$

Then we rewrite (32) as

$$\begin{aligned} \tilde{\phi}(\lambda_{m,t}) = & \sum_{n \in N_{m,t}} f_t(n) \cdot z_{m,n,t}^{\text{DS}} \cdot Q_{n,t} \\ & - \lambda_{m,t} \cdot \sum_{n \in N_{m,t}} \varepsilon^{\text{MEC}} \cdot \tilde{\mu}_{m,n,t} \cdot Q_{n,t} \cdot F_{n,t}. \end{aligned} \quad (35)$$

According to (35), we obtain a Dinkelbach Iterative Method based **COD-DS** (**DIM-COD-DS**) problem as

$$\begin{aligned} \text{DIM-COD-DS : } & \max_{z^{\text{DS}}, \tilde{\mu}} \tilde{\phi}(\lambda_{m,t}) \\ \text{s.t.} & \quad \text{C1, C3_1, C4, C5, C6,} \\ & \quad \text{C9 : } \tilde{\mu}_{m,n,t} = c_{m,n,t} \cdot c^{\text{max}}, \\ & \quad \forall n \in \mathcal{N}_{m,t}. \end{aligned}$$

Such a problem can be solved iteratively. To be specific, we initialize $\lambda_{m,t}$ and bring it into the **DIM-COD-DS** to obtain the optimal decision variable $(z_{m,t}^{\text{DS}})^*$ and $\tilde{\mu}^*$ under it. Then, we substitute these obtained decision variables into (29) to update the value of $\lambda_{m,t}$ and solve **DIM-COD-DS** problem again to calculate new decision variables. The iteration will stop until $0 \leq \tilde{\phi}(\lambda_{m,t}^*) \leq \xi$, where ξ is a small positive value and then we will obtain the optimal $(z_{m,t}^{\text{DS}})^*$ and $c_{m,t}^*$.

The **COD-DT** problem is solved by similar method and then we will obtain the optimal $(z_{m,t}^{\text{DT}})^*$. Then the reward $r_{m,t}$ is calculated based on (28).

D. Complexity Analysis

In this section, we will analyze the computational complexity of solving computation offloading decision problem, which contains a **COD-DS** problem and a **COD-DT** problem. Since these two problems have similar expressions, we take **COD-DS** problem as an example to show its computational complexity, which can be easily extended to the **COD-DT** problem.

Considering the fractional objective function of **COD-DS** problem, we employ a Dinkelbach method to solve it iteratively. The **DIM-COD-DS** problem with (35) as the objective function is a linear integer programming problem under a given $\lambda_{m,t}$, and it can be solved by a relax-and-fix heuristic algorithm [49]. The complexity of it can be expressed as $O(G \cdot n_1^3)$ [51], where G is the relax-and-fix iteration time and n_1 is the number of variables in the **DIM-COD-DS** problem. We assume that the iteration time for solving the **DIM-COD-DS** problem is U (in general, the Dinkelbach Method has super-linear convergence speed [50], which means

Algorithm 1 Optimization-Embedding Multi-Agent Dueling Deep Q Network

- 1: **Initialize:** Initialize $J, \gamma, \epsilon, \theta, \theta', \alpha, \alpha', \beta, \beta'$.
- 2: **for** episode = 1, ..., Ψ **do**
- 3: **Initialize:** Set $\mathbf{b}_0 = \{0, 0, \dots, 0\}$ and the state of all UAVs $S = \{s_1, s_2, \dots, s_M\}$
- 4: **for** $t = 1, 2, \dots$ **do**
- 5: Choose an action under current state $s_{m,t}$ based on ϵ -greedy policy.
- 6: Execute the feasible action $a_{m,t}$ to arrive at the location $l_{m,t}$ and collect the offloading requirements of users $n \in \mathcal{N}_{m,t}$.
- 7: Obtain the optimal admission control $(z_{m,t}^{\text{DS}})^*$, $(z_{m,t}^{\text{DT}})^*$ and computing resource allocation $c_{m,t}^*$ by solving **COD-DS** problem and **COD-DT** problem.
- 8: Calculate the reward $r_{m,t}$ by $(z_{m,t}^{\text{DS}})^*$, $(z_{m,t}^{\text{DT}})^*$, $c_{m,t}^*$ and obtain the next state $s_{m,t+1}$.
- 9: Store the transition tuple $(s_t, a_{m,t}, r_{m,t}, s_{m,t+1})$ into J .
- 10: Sample random minibatch of tuples j from J to train network.
- 11: Calculate the loss function by (24) and update the parameters of the main network.
- 12: **if** $\frac{t}{C} == 0$ **then**
- 13: Copy the main network parameters to the target network.
- 14: **end if**
- 15: Update ϵ .
- 16: **end for**
- 17: **end for**

the value of U is small). Assume that there are $N_{m,t}^{\text{DS}}$ users with delay sensitive tasks within the coverage of UAV m . So, parameter n_1 can be calculated as $n_1 = 2|N_{m,t}^{\text{DS}}|$. Thus, the computational complexity of **COD-DS** problem in time slot t can be approximately presented as $UO((N_{m,t}^{\text{DS}})^3)$.

Similarly, **COD-DT** problem is also solved by a Dinkelbach Iterative Method. It takes $z_{m,n,t}^{\text{DT}}$ as the decision variables and the number of them is $|N_{m,t}^{\text{DT}}|$. So the overall computational complexity of **COD-DS** problem and **COD-DT** problem in time slot t are approximately presented as

$$UO((N_{m,t}^{\text{DS}})^3) + UO((N_{m,t}^{\text{DT}})^3). \quad (36)$$

E. Optimization-Embedding Multi-Agent Dueling Deep Q Network Algorithm

To improve the performance of learning, we use the Dueling Deep Q Network (DDQN). In contrast to the DQN algorithm, DDQN replaces the output layer of DNN with two parallel fully connected layers with parameters α and β . Hence, the output of the main network contains two parts, namely, state value function $V(s_t; \theta, \beta)$ and advantage function $A(s_t, a_t; \theta, \alpha)$. The specific expression of the Q value function is

$$Q(s_t, a_t; \theta, \alpha, \beta) = V(s_t; \theta, \beta) + A(s_t, a_t; \theta, \alpha), \quad (37)$$

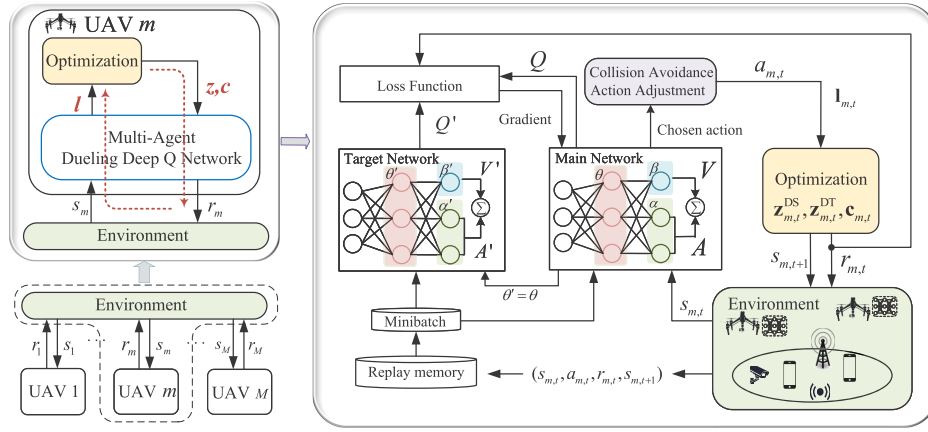


Fig. 3. OMADDQN algorithm framework.

Algorithm 2 Collision Avoidance Action Adjustment Mechanism

- 1: **Input:** The chosen action $a_{m,t}$ via ϵ -greedy policy.
- 2: Calculate the location $\hat{l}_{m,t}$ if UAV m executes $a_{m,t}$.
- 3: Calculate the distance $\hat{d}_{m,m',t}$ between the location of any other UAV $l_{m',t-1}$ and $\hat{l}_{m,t}$.
- 4: **if** $\hat{d}_{m,m',t} < D^{\min}$ after UAV m executing $a_{m,t}$ **then**
- 5: Adjust the action as $a_{m,t} = \{-\eta_{m,t}, v_{m,t}\}$.
- 6: Calculate the location $\tilde{l}_{m,t}$ of UAV m if it executes $a_{m,t} = \{-\eta_{m,t}, v_{m,t}\}$.
- 7: Calculate the distance $\tilde{d}_{m,m',t}$ between the location of other UAVs $l_{m',t-1}$ and $\tilde{l}_{m,t}$.
- 8: **if** $\tilde{d}_{m,m',t} < D^{\min}$ after UAV m executing $a_{m,t}$ **then**
- 9: Adjust the action as $a_{m,t} = \{0, 0\}$.
- 10: **end if**
- 11: **else**
- 12: Keep the chosen action $a_{m,t}$ unchanged.
- 13: **end if**
- 14: **Output:** The feasible action $a_{m,t}$.

where $V(s_t; \theta, \beta)$ reflects characteristics of current state, and $A(s_t, a_t; \theta, \alpha)$ reflects the advantage of action a_t compared to other actions. Thus, we can realize the relatively independent evaluation of the state and action in an environment and further highlight the advantages and disadvantages of each action to achieve better learning results [52]. However, in (37), one action value Q will correspond to many sets of state value function V and advantage function A . Hence, we change the representation of Q as

$$Q(s_t, a_t; \theta, \alpha, \beta) = V(s_t; \theta, \beta) + (A(s_t, a_t; \theta, \alpha) - \frac{1}{|A|} \sum_{a_{t'}} A(s_t, a_{t'}; \theta, \alpha)), \quad (38)$$

which does not change the essence of state value function and advantage function.

UAVs are equipped with a main network and a target network to train locally. The process of the OMADDQN algorithm is shown in Fig. 3. Note that one episode is the time from when UAVs take off until one of them returns to the

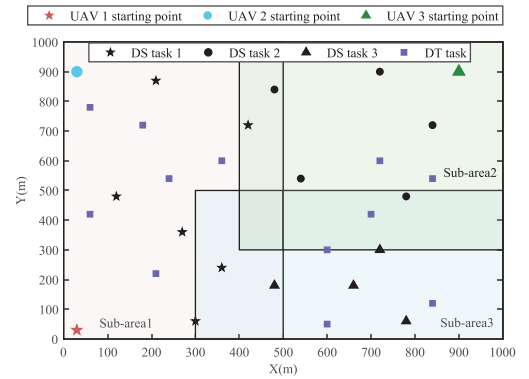


Fig. 4. The simulation scenario.

charging station. During each time slot, UAVs work following an F-S procedure as introduced in Section III. At the beginning of time slot t , UAV m obtains its state $s_{m,t}$ from BS [53] and utilizes ϵ -greedy policy to choose an action according to the Q value output by the main network. Then UAV m will execute its feasible action $a_{m,t}$ to fly to $l_{m,t}$. After that, each UAV will hover and get the optimal admission control $(z_{m,t}^{DS})^*$, $(z_{m,t}^{DT})^*$ and resource allocation $c_{m,t}^*$ by solving COD-DS problem and COD-DT problem, which are used to calculate the reward $r_{m,t}$. After that, UAV m gets the next state $s_{m,t+1}$ and stores the experience tuple $(s_{m,t}, a_{m,t}, r_{m,t}, s_{m,t+1})$ into the replay memory J .

Once the replay memory J is full, new tuples will replace the previous ones, and DNNs will start to be trained by randomly sampling minibatch tuples j from the replay memory. For each piece of tuple, $s_{m,t}$ and $a_{m,t}$ are fed into the main network to calculate the Q value, and the $r_{m,t}$ and $s_{m,t+1}$ are used to obtain the target Q value as (23). According to (24), the loss function is calculated by the Q -value and the target Q value, which will be used to update the parameters θ, α, β of the main network by performing a gradient step. Finally, parameters θ', α', β' of the target network are copied by the parameters from the main network every C time slots. The algorithm process is shown in Algorithm 1.

F. Collision Avoidance Action Adjustment Mechanism

Since there is no constraint on the chosen action that will lead to a collision, the action feasibility of the main

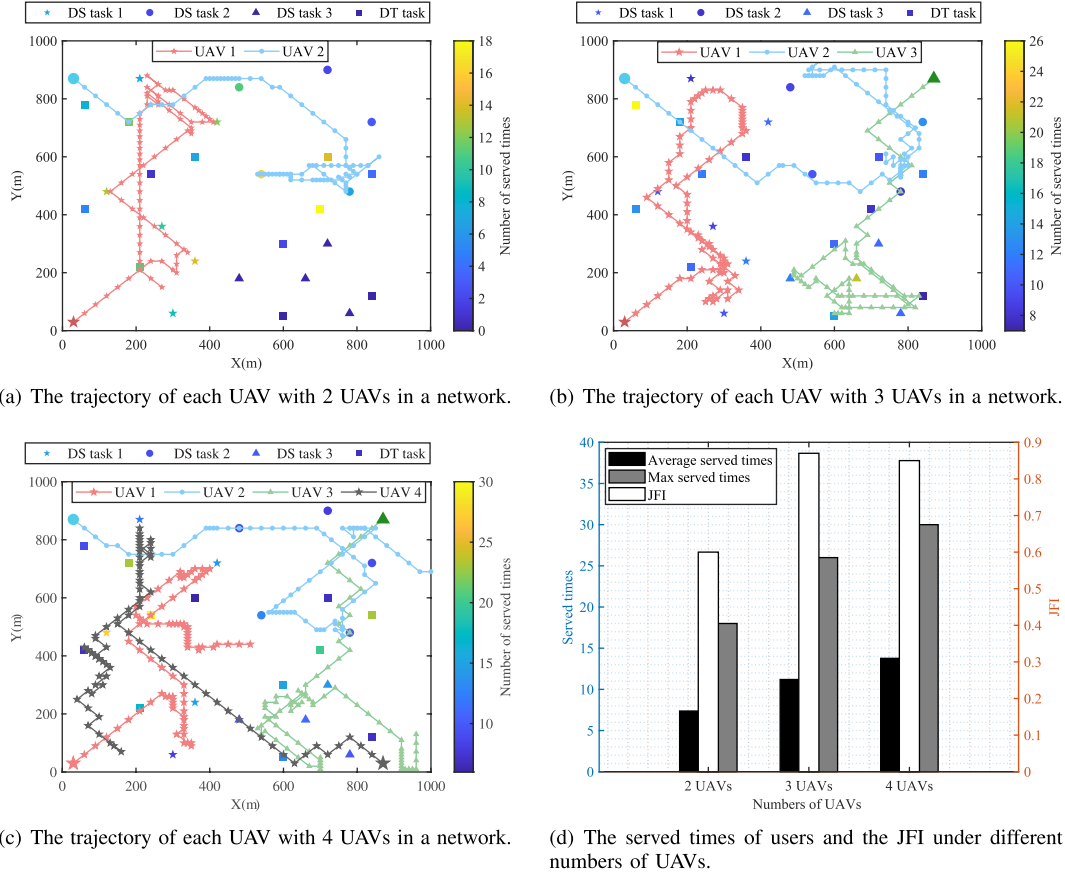


Fig. 5. The performance under different numbers of UAVs.

network output must be assessed to avoid the collision. We put forward a Collision Avoidance Action Adjustment Mechanism, as shown in **Algorithm 2**. In particular, during time slot t , UAV m firstly selects an action $a_{m,t}$. It is necessary to determine whether the distance between UAV m and any other UAV is less than a safe distance $D^{\min}$ after it performs the selected action $a_{m,t}$. If that is less than $D^{\min}$, it means executing $a_{m,t}$ will cause a collision. Then UAV m will change the chosen action to $a_{m,t} = \{-\eta_{m,t}, v_{m,t}\}$. If the adjusted action does not cause a collision, it will be executed. Otherwise, UAV m will hover, i.e., $a_{m,t} = \{0, 0\}$.

V. SIMULATION RESULTS AND DISCUSSIONS

A. Simulation Setup

We consider a UAV-assisted MEC network consisting of $M = 3$ UAVs [22], [36] and $N = 27$ users in a 1000×1000 m^2 area [25], [36], as shown in Fig. 4. Each user produces one task at the beginning of each time slot and all users are deployed randomly, 12 of which produce delay tolerant tasks, and others produce delay sensitive tasks. Delay sensitive tasks are divided into 3 types. There are 12 users with delay tolerant task randomly deployed following an uniform distribution in the whole service area, while 15 users with delay sensitive task are randomly deployed in three sub-areas, respectively, as shown in Fig. 4. Note that the task data volumes and requirements of user n are different in different time slots. Each UAV has one function and can only compute one

type of delay sensitive tasks. The initial location of each UAV is $[0, 0, 50]$, $[0, 900, 50]$, and $[900, 900, 50]$, respectively. We set that there are $\Psi = 1000$ episodes during the whole operation. We define one episode as the duration from take-off of UAVs until any of them return to the charging station. Each UAV takes off with battery capacity as $E^{\max} = 3250$ mAh (corresponding to 152100 J under 13 V) and returns to the charging station before energy is lower than $E^{\min} = 565$ mAh (corresponding to 26442 J under 13 V) [55]. Other parameter settings are presented in Table III.

Furthermore, to demonstrate the fair service of UAVs, we introduce a metric called Jain Fairness Index (JFI) [56], which is expressed as

$$F = \frac{(\sum_{n=1}^N b_n)^2}{N \cdot \sum_{n=1}^N (b_n)^2}, \quad (39)$$

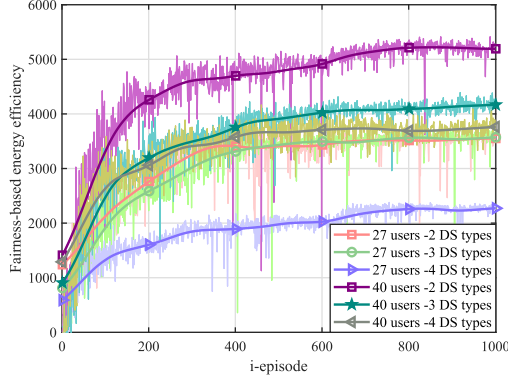
where b_n represents the times user n has been served during a whole episode.

B. Results and Analysis

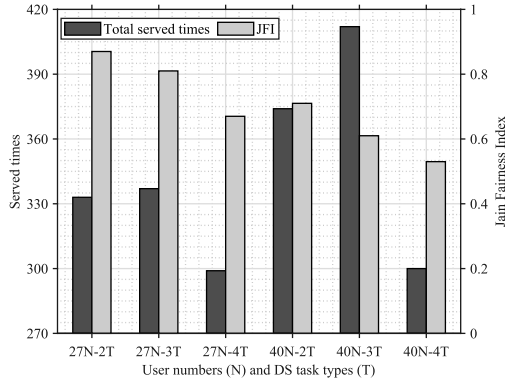
First, to demonstrate the effectiveness of the proposed MA-ETCO scheme, we evaluate the performance of our scheme under different numbers of UAVs in Fig. 5, which illustrates trajectory of each UAV as well as service times and JFI of users. Fig. 5 (a) shows that each UAV will fly to the

TABLE III
PARAMETER SETTINGS

Parameters	Values
Discount rate, γ	0.9 [52], [54], [55]
Mini-batch size, j	128
Replay memory size, J	10000 [55]
Flight speed of UAV, $v_{m,t}$	{0,10,20,30} m/s [38], [44]
Coverage radius of UAV, R^{cov}	80 m [32]
Channel bandwidth, B	10 MHz [32], [36]
Transmission power of users and UAVs, P_n, P_m	0.1 W, 1 W [35], [36]
Variance of the white Gaussian noise, σ^2	5×10^{-15}
Effective capacitance coefficient of UAV, e^{MEC}	10^{-28} [47], [48]
Data volume of task produced by user n , $D_{n,t}$	[5,12] MB
The number of CPU cycles for one bit, $F_{n,t}$	[500,2000] cycles/bit [36], [37]
The maximum CPU frequencies for each UAV, c^{max}	10^{10} cycles/s [47], [48]
Minimal safe distance between UAVs, D^{min}	10 m



(a) The fairness-based energy efficiency under different numbers of users and delay sensitive task types.



(b) The served times of users and JFI under different numbers of users and delay sensitive task types.

Fig. 6. The performance under different number of users and different numbers of delay sensitive task types.

area with more matching delay sensitive and delay tolerant tasks, and when there are only two UAVs, some users are not served, leading to a ‘service blind spot’ in a network. The result indicates that if the number of delay sensitive task types exceeds the number of types that all UAVs can handle, some users will not be served. Therefore, when there are three or four UAVs in a network, they well cover service areas and all users are served as shown in Fig. 5 (b)-(c), and four UAVs will serve more users in one episode, as shown in Fig. 5 (d).

Second, to evaluate the performance of our scheme under different numbers and types of tasks, we present the fairness-based energy efficiency, total served times, and JFI of users with 3 UAVs in Fig. 6. When user density increases, the number of users within the coverage of each UAV will also increase. Thus, UAVs will offload more tasks in each time slot

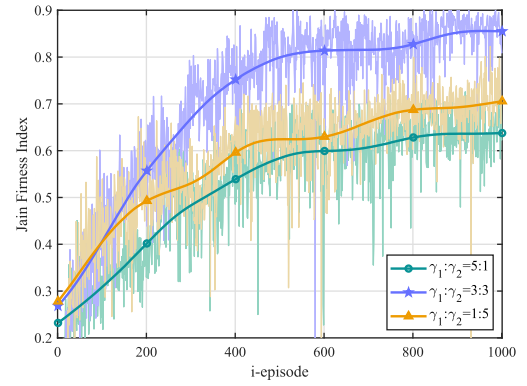


Fig. 7. The JFI under different weight ratios $\gamma_1 : \gamma_2$.

and achieve higher fairness-based energy efficiency, as shown in Fig. 6 (a). However, the JFI is low in scenarios with more users because UAVs have limited energy and cannot serve all users in one episode. Therefore, they tend to serve the areas with high user density in priority and ignore some remote users, leading to a low JFI. Furthermore, if the number of delay sensitive task types exceeds the number of types that all UAVs can handle, some delay sensitive tasks cannot be served by any UAV, leading to a low JFI, as shown in Fig. 6 (b).

Third, to demonstrate the impact of different serving priorities on delay sensitive or delay tolerant tasks on fairness, we show the JFI of our proposed scheme under different weight ratios $\gamma_1 : \gamma_2$ in Fig. 7. When the weights of serving delay sensitive and delay tolerant tasks are similar, such as $\gamma_1 : \gamma_2 = 3 : 3$, the JFI reaches about 0.85 indicating that the proposed scheme will guide UAVs to ensure fairness. By changing the weight ratio, such as $\gamma_1 : \gamma_2 = 5 : 1$ or $\gamma_1 : \gamma_2 = 1 : 5$, we can adjust the serving priority on delay sensitive or delay tolerant tasks.

In Fig. 8, we show trajectory of each UAV without taking fairness into account by setting $f_t(n) = 1$ in the reward (28). Compared with Fig. 5 (b), each UAV without considering the fairness keeps flying around a specific area. So, many users are not served and the JFI is about 0.3.

Next, to demonstrate the importance of joint design on computation offloading energy efficiency, we compare the proposed MA-ETCO scheme with three other schemes, namely, MA-ETO scheme, MA-TCO scheme, and MA-TO scheme.

- MA-ETO: In this scheme, only trajectory control and admission control are considered. The computation resources of a UAV are equally allocated to admitted delay sensitive tasks.
- MA-TCO: In this scheme, the objective is to maximize the service data volumes of UAVs without considering energy consumption.
- MA-TO: This scheme is built on the MA-ETO scheme and changes the objective to maximize the service data volumes of UAVs without considering energy consumption.

In Fig. 9, we show the average fairness-based energy efficiency of three UAVs under four schemes above. As shown in Fig. 9, UAVs under the MA-ETCO scheme achieve the highest energy efficiency. Compared to the MA-TCO scheme, UAVs

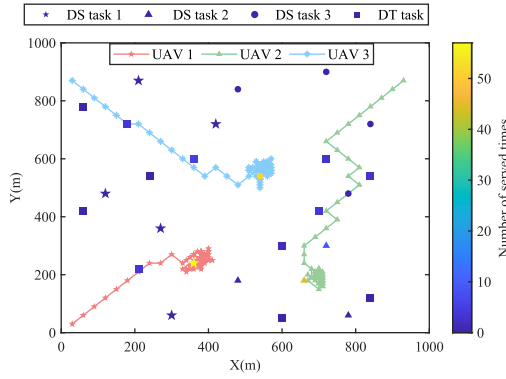


Fig. 8. The trajectories of UAVs without considering the fairness, i.e., $f_t(n) = 1$.

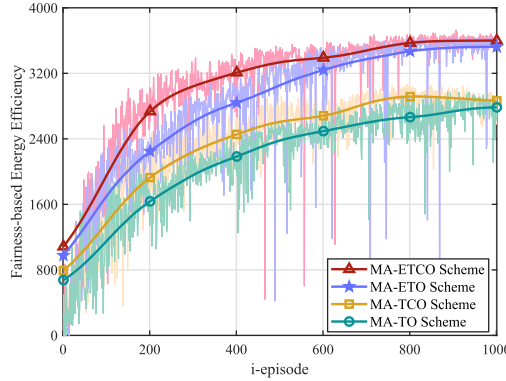


Fig. 9. The fairness-based computation offloading energy efficiency of different schemes.

under the MA-ETCO scheme tend to minimize flight and computation energy consumption while serving. Furthermore, by comparing with the MA-ETO scheme, UAVs under the MA-ETCO scheme allocate reasonable computation resources for admitted tasks to minimize the computation energy consumption with latency guarantee and prevent the waste of resources.

Then, we compare the performance of the proposed OMADDQN algorithm with four other algorithms, namely, OSADDQN, OMADQN, MADDQN, and RANDOM.

- Optimization-Embedding Single-Agent Dueling Deep Q Network (OSADDQN) Algorithm: Under this algorithm, UAVs independently learn their strategy without others' state information.
- Optimization-Embedding Multi-Agent Deep Q Network (OMADQN) Algorithm: This algorithm uses the DQN network, which lacks an advantage function compared to the proposed algorithm.
- Multi-Agent Dueling Deep Q Network (MADDQN) Algorithm: This algorithm uses deep reinforcement learning to learn all decisions, including UAV trajectory control, admission control, and computation resource allocation, where the continuous computation resource allocation decision variables are discretized into five levels.
- RANDOM: In this algorithm, UAVs randomly fly around a network.

In Fig. 10, we show the fairness-based energy efficiency under these algorithms. It is obvious that our proposed

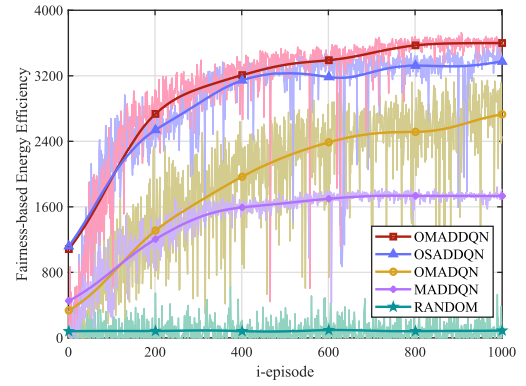


Fig. 10. Fairness-based energy efficiency under different algorithms.

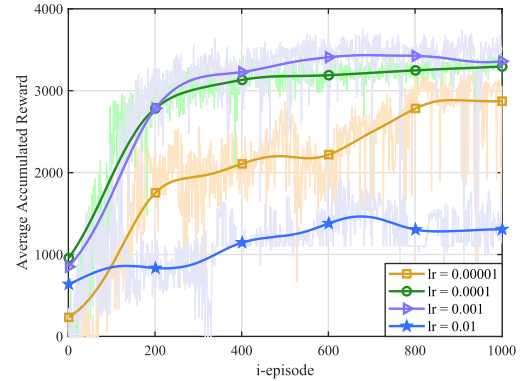


Fig. 11. The average accumulate reward under different learning rates.

OMADDQN has the best performance. Compared with OMADQN, the OMADDQN algorithm uses an advantage function to highlight the advantages of each action, which allows UAVs to select the best action frequently and accurately and thus achieve better FEE performance. Besides, the FEE curve of OMADDQN is more stable than that of OMADQN. As for the OSADDQN algorithm, it cannot achieve optimal global performance because each UAV independently learns its strategy without considering others' strategies. As for the MADDQN algorithm, the discretization of continuous decision variables may cause the obtained action to not be optimal. Furthermore, the large action space dimension leads to slower convergence.

Finally, we investigate the impact of different learning rates on the average accumulated reward of UAVs and present the results in Fig. 11. The learning rate determines the step size for neural network updating. We can see that the average accumulated reward converges poorly when the learning rate is too large or too small. A large learning rate causes the gradient to oscillate around the minimum value and prevents finding the optimal solution. When the learning rate is too small, the convergence process will become very slow and may fall into a local optimum.

VI. CONCLUSION

In this paper, to maximize the fairness-based computation offloading energy efficiency of a UAV-assisted MEC network, we propose an MA-ETCO scheme. We jointly optimize the trajectories and computation offloading decisions of UAVs with consideration on various task types required from users

and various functionalities of different UAVs enabled by different application programs. To adapt to a dynamic environment, we develop an OMADDQN algorithm. Compared to relying solely on deep reinforcement learning, such an optimization-embedding way reduces action space dimension and improves convergence efficiency. Simulation results show that UAVs in the MA-ETCO scheme under the OMADDQN algorithm achieve high MEC service efficiency.

REFERENCES

- [1] X. Du, X. Li, N. Zhao, and X. Wang, "A joint trajectory and computation offloading scheme for UAV-MEC networks via multi-agent deep reinforcement learning," in *Proc. IEEE Int. Conf. Commun.*, Rome, Italy, Jun. 2023, pp. 5438–5443.
- [2] X. Li, R. Xiao, M. Pan, F. Jiang, N. Zhao, and X. Wang, "Green traffic off-loading over uncertain shared spectrums with end-to-end QoS guarantee," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 9921–9937, Sep. 2020.
- [3] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1584–1607, Aug. 2019.
- [4] S. Zhu, W. Xu, L. Fan, K. Wang, and G. K. Karagiannis, "A novel cross entropy approach for offloading learning in mobile edge computing," *IEEE Wireless Commun. Lett.*, vol. 9, no. 3, pp. 402–405, Mar. 2020.
- [5] X. Wang et al., "Dynamic resource scheduling in mobile edge cloud with cloud radio access network," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 11, pp. 2429–2445, Nov. 2018.
- [6] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, London, U.K., Jul. 2020, pp. 2076–2085.
- [7] K. Wang, L. Wang, C. Pan, and H. Ren, "Deep reinforcement learning-based resource management for flexible mobile edge computing: Architectures, applications, and research issues," *IEEE Veh. Technol. Mag.*, vol. 17, no. 2, pp. 85–93, Jun. 2022.
- [8] C. Xu, Q. Xu, J. Wang, K. Wu, K. Lu, and C. Qiao, "AoI-centric task scheduling for autonomous driving systems," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, London, U.K., May 2022, pp. 1019–1028.
- [9] S. Raj Pokhrel and J. Choi, "Adaptive coexistence of delay-sensitive and delay-tolerant MTC devices: A control-theoretic approach," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16017–16027, Sep. 2022.
- [10] K. G. Panda, S. Das, and D. Sen, "Efficient UAV placement strategy for guaranteed QoS demand," in *Proc. IEEE 92nd Veh. Technol. Conf. (VTC-Fall)*, Nov. 2020, pp. 1–5.
- [11] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.
- [12] Y. Liu, W. Huangfu, H. Zhou, H. Zhang, J. Liu, and K. Long, "Fair and energy-efficient coverage optimization for UAV placement problem in the cellular network," *IEEE Trans. Commun.*, vol. 70, no. 6, pp. 4222–4235, Jun. 2022.
- [13] S. Sun, G. Zhang, H. Mei, K. Wang, and K. Yang, "Optimizing multi-UAV deployment in 3-D space to minimize task completion time in UAV-enabled mobile edge computing systems," *IEEE Commun. Lett.*, vol. 25, no. 2, pp. 579–583, Feb. 2021.
- [14] Q. Luan, H. Cui, L. Zhang, and Z. Lv, "A hierarchical hybrid subtask scheduling algorithm in UAV-assisted MEC emergency network," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12737–12753, Jul. 2022.
- [15] M. Peer, V. A. Bohara, A. Srivastava, and G. Ghatak, "User mobility-aware time stamp for UAV-BS placement," in *Proc. IEEE Wireless Commun. Netw. Conf. Workshops (WCNCW)*, Mar. 2021, pp. 1–6.
- [16] R. Chen, Y. Sun, L. Liang, and W. Cheng, "Joint power allocation and placement scheme for UAV-assisted IoT with QoS guarantee," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 1066–1071, Jan. 2022.
- [17] X. He, R. Jin, and H. Dai, "Joint service placement and resource allocation for multi-UAV collaborative edge computing," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2021, pp. 1–6.
- [18] G. K. Xilouris, M. C. Batistatos, G. E. Athanasiadou, G. Tsoulos, H. B. Pervaiz, and C. C. Zarakovitis, "UAV-assisted 5G network architecture with slicing and virtualization," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2018, pp. 1–7.
- [19] A. Roy and P. Bouvry, "Opti-U: Optimal UAV selection for enabling UAV-as-a-service," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 1–7.
- [20] N. Pathak, S. Misra, A. Mukherjee, A. Roy, and A. Y. Zomaya, "UAV virtualization for enabling heterogeneous and persistent UAV-as-a-service," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 6731–6738, Jun. 2020.
- [21] Y. Xu, T. Zhang, J. Loo, D. Yang, and L. Xiao, "Completion time minimization for UAV-assisted mobile-edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12253–12259, Nov. 2021.
- [22] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "Cellular-connected multi-UAV MEC networks: An online stochastic optimization approach," *IEEE Trans. Commun.*, vol. 70, no. 10, pp. 6630–6647, Oct. 2022.
- [23] Z. Yang, S. Bi, and Y. A. Zhang, "Online trajectory and resource optimization for stochastic UAV-enabled MEC systems," *IEEE Trans. Wireless Commun.*, vol. 21, no. 7, pp. 5629–5643, Jul. 2022.
- [24] B. Liu, Y. Wan, F. Zhou, Q. Wu, and R. Q. Hu, "Resource allocation and trajectory design for MISO UAV-assisted MEC networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4933–4948, May 2022.
- [25] Y. Luo, W. Ding, and B. Zhang, "Optimization of task scheduling and dynamic service strategy for multi-UAV-enabled mobile-edge computing system," *IEEE Trans. Cognit. Commun. Netw.*, vol. 7, no. 3, pp. 970–984, Sep. 2021.
- [26] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "UAV-assisted MEC networks with aerial and ground cooperation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 12, pp. 7712–7727, Dec. 2021.
- [27] H. Mei, K. Yang, Q. Liu, and K. Wang, "Joint trajectory-resource optimization in UAV-enabled edge-cloud system with virtualized mobile clone," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5906–5921, Jul. 2020.
- [28] J. Ji, K. Zhu, C. Yi, and D. Niyato, "Energy consumption minimization in UAV-assisted mobile-edge computing systems: Joint resource allocation and trajectory design," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8570–8584, May 2021.
- [29] J. Li, C. Yi, J. Chen, K. Zhu, and J. Cai, "Joint trajectory planning, application placement, and energy renewal for UAV-assisted MEC: A triple-learner-based approach," *IEEE Internet Things J.*, vol. 10, no. 15, pp. 13622–13636, Aug. 2023.
- [30] Y. Liu, J. Yan, and X. Zhao, "Deep reinforcement learning based latency minimization for mobile edge computing with virtualization in maritime UAV communication network," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4225–4236, Apr. 2022.
- [31] S. Wan, J. Lu, P. Fan, and K. B. Letaief, "Toward big data processing in IoT: Path planning and resource management of UAV base stations in mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5995–6009, Jul. 2020.
- [32] H. Wang, H. Ke, and W. Sun, "Unmanned-aerial-vehicle-assisted computation offloading for mobile edge computing based on deep reinforcement learning," *IEEE Access*, vol. 8, pp. 180784–180798, 2020.
- [33] Y. Liu, S. Xie, and Y. Zhang, "Cooperative offloading and resource management for UAV-enabled mobile edge computing in power IoT system," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12229–12239, Oct. 2020.
- [34] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.
- [35] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
- [36] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cognit. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.
- [37] A. M. Seid, G. O. Boateng, S. Anokye, T. Kwantwi, G. Sun, and G. Liu, "Collaborative computation offloading and resource allocation in multi-UAV-assisted IoT networks: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 12203–12218, Aug. 2021.
- [38] W. Liu, B. Li, W. Xie, Y. Dai, and Z. Fei, "Energy efficient computation offloading in aerial edge networks with multi-agent cooperation," *IEEE Trans. Wireless Commun.*, vol. 22, no. 9, pp. 5725–5739, Sep. 2023.
- [39] Y. Liu, Z. Cao, H. Xiong, J. Du, H. Cao, and L. Zhang, "Dynamic obstacle avoidance for cable-driven parallel robots with mobile bases via sim-to-real reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 8, no. 3, pp. 1683–1690, Mar. 2023.

- [40] K. Zhu, B. Li, W. Zhe, and T. Zhang, "Collision avoidance among dense heterogeneous agents using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 57–64, Jan. 2023.
- [41] N. Pathak, A. Mukherjee, and S. Misra, "AerialBlocks: Blockchain-enabled UAV virtualization for industrial IoT," *IEEE Internet Things Mag.*, vol. 4, no. 1, pp. 72–77, Mar. 2021.
- [42] H. Cao, S. Wu, Y. Hu, G. S. Augla, and L. Yang, "Virtual resource allocation for tactile and flexible services in UAVs-integrated 5G networks," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2020, pp. 1–6.
- [43] Z. Wang, R. Liu, Q. Liu, L. Han, and J. S. Thompson, "Feasibility study of UAV-assisted anti-jamming positioning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 7718–7733, Aug. 2021.
- [44] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [45] A. Goldsmith, *Wireless Communications*. New York, NY, USA: Cambridge Univ. Press, 2005.
- [46] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.
- [47] H. Hu, W. Song, Q. Wang, R. Q. Hu, and H. Zhu, "Energy efficiency and delay tradeoff in an MEC-enabled mobile IoT network," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15942–15956, Sep. 2022.
- [48] Y. Ye, L. Shi, H. Sun, R. Q. Hu, and G. Lu, "System-centric computation energy efficiency for distributed NOMA-based MEC networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8938–8948, Aug. 2020.
- [49] M. Pan, C. Zhang, P. Li, and Y. Fang, "Spectrum harvesting and sharing in multi-hop CRNs under uncertain spectrum supply," *IEEE J. Sel. Areas Commun.*, vol. 30, no. 2, pp. 369–378, Feb. 2012.
- [50] Y. Yu, X. Bu, K. Yang, Z. Wu, and Z. Han, "Green large-scale fog computing resource allocation using joint benders decomposition, dinkelbach algorithm, ADMM, and branch-and-bound," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4106–4117, Jun. 2019.
- [51] Y. Ye, "An $O(n^3L)$ potential reduction algorithm for linear programming," *Math. Program.*, vol. 50, no. 1, pp. 239–258, 1991.
- [52] C. Qiu, F. R. Yu, H. Yao, C. Jiang, F. Xu, and C. Zhao, "Blockchain-based software-defined industrial Internet of Things: A dueling deep Q-learning approach," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4627–4639, Jun. 2019.
- [53] F. Wu, H. Zhang, J. Wu, Z. Han, H. V. Poor, and L. Song, "UAV-to-device underlay communications: Age of information minimization by multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 69, no. 7, pp. 4461–4475, Jul. 2021.
- [54] Y. Nie, J. Zhao, F. Gao, and F. R. Yu, "Semi-distributed resource management in UAV-aided MEC systems: A multi-agent federated reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13162–13173, Dec. 2021.
- [55] X. Li, S. Cheng, H. Ding, M. Pan, and N. Zhao, "When UAVs meet cognitive radio: Offloading traffic under uncertain spectrum environment via deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 824–838, Feb. 2023.
- [56] Z. Yang, W. Xu, Y. Pan, C. Pan, and M. Chen, "Optimal fairness-aware time and power allocation in wireless powered communication networks," *IEEE Trans. Commun.*, vol. 66, no. 7, pp. 3122–3135, Jul. 2018.



Xuanheng Li (Member, IEEE) received the Ph.D. degree in communication and information systems from the Dalian University of Technology, Dalian, China, in 2018. From 2015 to 2017, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Florida. He is currently an Associate Professor with the Dalian University of Technology. His current research interests include intelligent communications, wireless sensing, UAV-assisted networks, and mobile edge computing and caching. He was a recipient of the Best Paper Award at the 2015 IEEE Globecom.



Xinyang Du (Student Member, IEEE) received the B.S. degree from the Dalian University of Technology, China, where she is currently pursuing the master's degree with the School of Information and Communication Engineering. Her current research interests include UAV-assisted networks and mobile edge computing.



Nan Zhao (Senior Member, IEEE) received the Ph.D. degree in information and communication engineering from the Harbin Institute of Technology, Harbin, China, in 2011. He is currently a Professor with the Dalian University of Technology, China. He won the best paper awards in IEEE VTC in Spring 2017, ICNC in 2018, WCSP in 2018, and WCSP in 2019. He received the IEEE Communications Society Asia-Pacific Board Outstanding Young Researcher Award in 2018. He is serving on the editorial boards of IEEE WIRELESS COMMUNICATIONS, IEEE WIRELESS COMMUNICATIONS LETTERS, and IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING.



Xianbin Wang (Fellow, IEEE) received the Ph.D. degree in electrical and computer engineering from the National University of Singapore in 2001. He is currently a Professor and a Tier-1 Canada Research Chair of 5G and wireless IoT communications with Western University, Canada. Prior to joining Western University, he was with the Communications Research Centre Canada as a Research Scientist/Senior Research Scientist from 2002 to 2007. From 2001 to 2002, he was a System Designer with STMicroelectronics. His current research interests include 5G/6G technologies, the Internet of Things, communications security, machine learning, and intelligent communications. He has over 500 highly cited journals and conference papers, in addition to more than 30 granted and pending patents and several standard contributions. He is a fellow of the Canadian Academy of Engineering and the Engineering Institute of Canada. He has received many prestigious awards and recognitions, including the IEEE Canada R. A. Fessenden Award, the Canada Research Chair, the Engineering Research Excellence Award at Western University, the Canadian Federal Government Public Service Award, the Ontario Early Researcher Award, and nine best paper awards. He was involved in many IEEE conferences, including GLOBECOM, ICC, VTC, PIMRC, WCNC, CCECE, and CWIT, in different roles, such as the General Chair, the TPC Chair, the Symposium Chair, the Tutorial Instructor, the Track Chair, the Session Chair, and a Keynote Speaker. He serves/has served as the editor-in-chief, an associate editor-in-chief, and an editor/associate editor for more than ten journals. He was the Chair of the IEEE ComSoc Signal Processing and Computing for Communications (SPCC) Technical Committee and is currently serving as the Central Area Chair for IEEE Canada.